

UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

DIPARTIMENTO DI INGEGNERIA
DELL'INFORMAZIONE

Corso di Laurea Magistrale in Ingegneria Informatica

Relazione Conclusiva

Elaborato del corso di Algoritmi e Strutture Dati

Studenti:
Bagnalasta Federico Matricola n. 000000
Kovacic Matteo Matricola n. 736588
Zambelli Samuele Matricola n. 736800

Anno Accademico 2024/2025

Indice

1 Introduzione **2**
1.1 Costrutti fondamentali 4

2 Compito 1 : Creazione della Griglia **8**

3 Compito 2: Calcolo di Contesto, Complemento e Frontiera **10**

4 Compito 3: Determinazione del Cammino Minimo Base **15**
4.1 Stop anticipato e Gestione dei Risultati Sub-Ottimi 18

5 Presentazione del Risultato **19**
5.1 Ottimizzazioni dell'Algoritmo Cammino Minimo 20

6 Compito 4: Analisi Prestazioni **22**
6.1 Analisi Spaziale 22
6.2 Analisi Temporale 23
6.3 Benchmark e Campione di Test 24

7 Conclusioni **25**

8 Manuale d'installazione e utilizzo **26**
8.1 Requisiti di Sistema 26
8.2 Avvio del Programma 26
8.3 Sintassi generale 27
8.4 Modalità Batch (CLI) 27
8.5 Output della Modalità Batch 28
8.6 Modalità Grafica (GUI) 31
8.7 Salvataggio e Caricamento 33
8.8 Ottimizzazioni dell'Algoritmo 34

1 Introduzione

La presente relazione descrive lo sviluppo di un'applicazione dedicata alla risoluzione dei requisiti presentati dall'elaborato del corso di Algoritmi e Strutture Dati a.a. 2024-2025.

Dopo un'approfondita analisi del problema e la definizione dei requisiti funzionali e non funzionali — già trattate in documentazione precedente — questa relazione intende fornire una sintesi delle scelte progettuali e implementative adottate, con particolare attenzione agli aspetti algoritmici, strutturali e prestazionali del sistema.

In primo luogo vengono illustrate le strutture dati selezionate per rappresentare la griglia, i nodi e le relazioni tra celle, motivando tali scelte alla luce delle esigenze di efficienza e manutenibilità. Successivamente, si discutono le principali strategie implementative adottate, evidenziando le soluzioni progettuali impiegate per ottimizzare l'esecuzione degli algoritmi di ricerca e di gestione dei percorsi.

La relazione prosegue con un'analisi delle ottimizzazioni introdotte sia a livello algoritmico sia a livello architetturale, seguita da una valutazione delle prestazioni ottenuta mediante misurazioni sperimentali e confronto tra differenti configurazioni.

Infine, viene presentato il manuale d'uso dell'applicazione, volto a chiarire le modalità operative, l'interfaccia fornita e le principali funzionalità disponibili.

L'obiettivo complessivo è fornire una visione chiara, strutturata e motivata dell'intero processo di progettazione e realizzazione del sistema, consentendo al lettore di comprendere sia le scelte tecniche effettuate sia i risultati conseguiti.

Sintesi dei requisiti del progetto

Il sistema oggetto della presente applicazione opera su uno spazio bidimensionale discretizzato in celle, all'interno del quale è possibile definire percorsi tra celle attraversabili. Le celle sono considerate adiacenti secondo il modello delle otto direzioni (cardinali e diagonali), con costi unitari associati alle mosse cardinali e costi pari a $\sqrt{2}$ per quelle diagonali. Le celle non attraversabili, se contigue, danno origine a ostacoli; resta tuttavia consentito il passaggio diagonale attraverso il solo contatto angolare tra due ostacoli.

Il progetto richiede innanzitutto la realizzazione di un generatore di griglie finite, configurabile dall'utente, in grado di produrre distribuzioni casuali di celle non attraversabili e diverse tipologie strutturate di ostacoli (semplici, aggregati, diagonali, delimitatori di aree chiuse, a barre e ulteriori conformazioni opportune). Tali griglie costituiscono la base dati per la successiva fase algoritmica.

Una seconda parte del lavoro riguarda lo sviluppo degli algoritmi necessari a determinare, data una cella origine, il relativo *contesto* e *complemento*, corrispondenti alle celle raggiungibili tramite cammini liberi di tipo 1 o di tipo 2. È inoltre richiesto il calcolo della distanza libera tra coppie di celle e l'individuazione di cammini minimi secondo

la definizione fornita, che include la possibilità di una costruzione ricorsiva mediante attraversamento delle celle di frontiera.

La terza componente del progetto consiste nell'estensione dell'applicazione affinché sia in grado di calcolare, quando esistente, la sequenza di *landmark* di un cammino minimo da una cella origine a una cella destinazione, nonché il cammino completo corrispondente. L'elaborazione deve tenere conto della possibilità che la cella destinazione non appartenga alla chiusura dell'origine, rendendo necessaria la composizione di più cammini minimi intermedi. La consegna contempla inoltre la possibilità di introdurre criteri euristici di ordinamento delle celle di frontiera, finalizzati al miglioramento dell'efficienza computazionale.

Infine, il progetto prevede una fase di sperimentazione sistematica, basata sulle griglie generate al primo punto, finalizzata alla valutazione delle prestazioni dell'algoritmo `CAMMINOMIN` e, ove pertinente, al confronto tra differenti strategie implementative ed euristiche. La sperimentazione deve essere condotta su tutte le tipologie di griglie prodotte dal generatore, così da garantire una valutazione completa e comparativa del comportamento dell'algoritmo al variare della struttura dello spazio di ricerca.

1.1 Costrutti fondamentali

I **costrutti principali** del progetto sono due: la **Cella** e la **Griglia**.

La **Cella** rappresenta il costrutto fondamentale del sistema ed è caratterizzata dai seguenti attributi:

- **riga** e **colonna**, che ne identificano la posizione all'interno della griglia;
- **tipo di cella**, che può assumere i valori di *ostacolo*, *complemento*, *contesto* oppure *libera*;
- un **valore booleano** che indica l'appartenenza o meno della cella alla *frontiera*.

La **Griglia** è invece costituita da un **array bidimensionale di celle** e include ulteriori informazioni strutturali, quali la cella di **Origine**, quella di **Destinazione** e la **tipologia della griglia**.

Tipologie di Griglia

La tipologia della griglia è rappresentata dall'enumerazione `TipoPattern`, che consente di generare automaticamente diversi pattern di distribuzione degli ostacoli. Le tipologie supportate sono le seguenti:

- **DENSITÀ_BASSA**, **DENSITÀ_MEDIA**, **DENSITÀ_ALTA**: griglie generate automaticamente con una densità di ostacoli pari rispettivamente a **0.10**, **0.25** e **0.45** rispetto al numero totale delle celle. Gli ostacoli vengono distribuiti in maniera casuale, secondo quanto descritto nel Capitolo 2.
- **PERSONALIZZATA**: griglia disegnata manualmente dall'utente tramite interfaccia grafica, che consente la massima libertà nella definizione della configurazione spaziale.
- **SPIRALE**, **SCACCHI**, **RIGHE**, **UNICO OSTACOLO**: pattern strutturati comunemente utilizzati, generati automaticamente dal sistema. Queste tipologie si sono rivelate particolarmente utili durante le fasi di *debugging* e *testing* del progetto; per tale motivo, è stata scelta la loro inclusione tra le opzioni disponibili per l'utente.
- **VUOTA**: griglia priva di ostacoli, utile come caso limite per la verifica del corretto funzionamento degli algoritmi di ricerca del cammino.

Esempi di Pattern

Di seguito sono riportate alcune immagini esemplificative delle diverse tipologie di griglia supportate dal sistema.

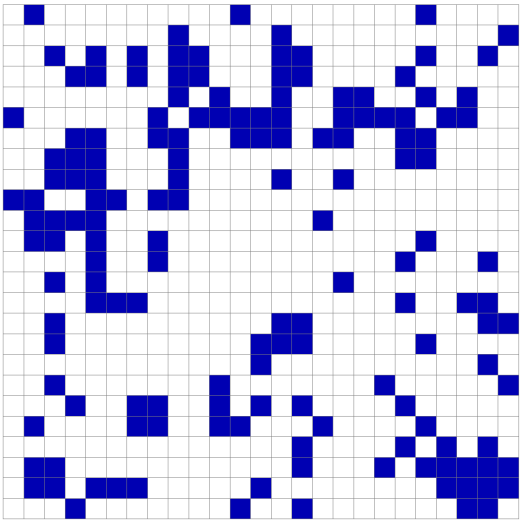


Figura 1: Esempio di griglia con pattern a spirale.

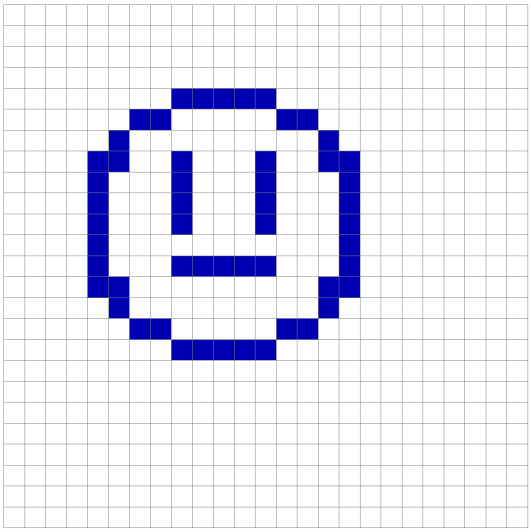
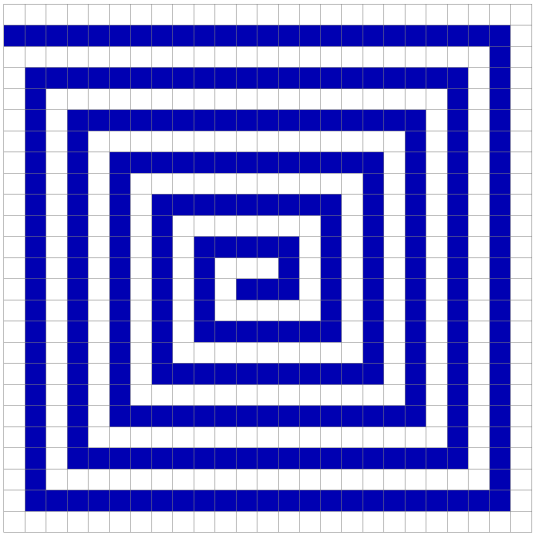
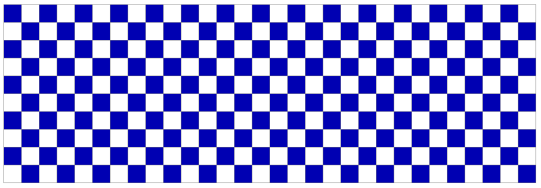


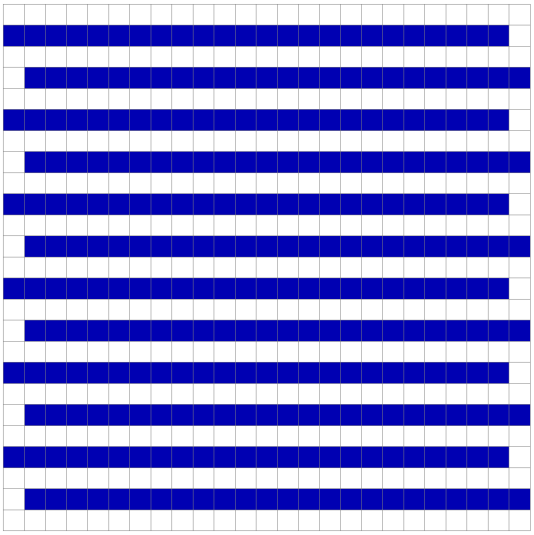
Figura 2: Esempio di griglia personalizzata definita dall'utente.



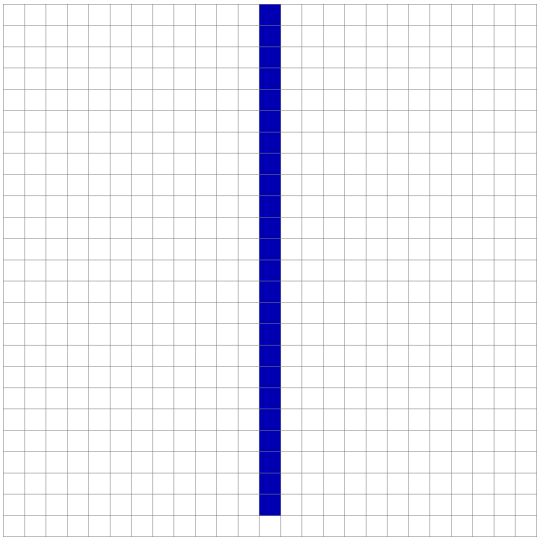
(a) Spirale



(b) Scacchi



(c) Righe



(d) Unico ostacolo

Figura 3: Esempi di pattern automatici.

All'interno del programma (eseguibile) è stata inoltre introdotta la possibilità di **serializzare l'oggetto Griglia in formato JSON**. Tale funzionalità consente di salvare e riutilizzare configurazioni precedenti, includendo implicitamente anche gli oggetti di tipo *Cella*. Questa scelta progettuale risulta particolarmente utile in fase di **testing**, poiché permette di memorizzare configurazioni specifiche di origine e destinazione, snellendo il processo di verifica. Inoltre, il riutilizzo delle stesse configurazioni favorisce la riproducibilità degli esperimenti e la coerenza dei risultati ottenuti.

Nota sulle dimensioni della griglia

L'oggetto **Griglia** può assumere dimensioni arbitrariamente diverse, non limitandosi esclusivamente a griglie quadrate (stesso numero di righe e colonne). Tuttavia, nella versione grafica, per motivi legati alle risorse di calcolo e alle prestazioni della macchina, la dimensione massima selezionabile è di 100×100 celle.

2 Compito 1 : Creazione della Griglia

La creazione della griglia è gestita tramite un'**interfaccia grafica** che permette all'utente di:

- definire le **dimensioni della griglia**;
- scegliere un **pattern predefinito** tra quelli descritti nel Capitolo 1;
- oppure selezionare il pattern **PERSONALIZZATO** e disegnare manualmente la propria griglia.

Al termine della fase di creazione, le celle possono assumere esclusivamente i valori di **ostacolo** o **libera**.

Creazione dei Pattern di Ostacoli

La generazione dei pattern di ostacoli dipende dal tipo di pattern scelto:

- **Pattern deterministici:** SPIRALE, RIGHE, UNICO_OSTACOLO e SCACCHI. Per questi pattern, gli ostacoli vengono posizionati seguendo algoritmi deterministici che stabiliscono la posizione di ciascuna cella ostacolo.
- **Pattern randomici:** DENSITÀ_BASSA, DENSITÀ_MEDIA, DENSITÀ_ALTA. In questo caso, l'utente può specificare un **seme** per il generatore casuale. Grazie alla funzione `Random` di Java, gli ostacoli vengono aggiunti in maniera randomica. La **densità target** del pattern oltre che hai rapporti numerici tra le varie tipologie di ostacoli base vengono utilizzate per calcolare il numero esatti di ostacoli base da aggiungere alla griglia. La randomicità opera su due livelli:
  1. **Scelta del rapporto tra le varie tipologie di ostacoli base(Semplici, Agglomerati, Diagonali, Barre; Anelli)**;
  2. **Selezione della cella di partenza per la generazione di ciascun ostacolo**.

Inoltre è possibile caricare file di tipo Json contenenti delle Griglie valide, precedentemente create.

Algoritmo di generazione degli ostacoli

Algorithm 1: Generazione Ostacoli

Input: Griglia di dimensione $M \times N$, tipo di pattern P, (opzionale) seme S
Output: Griglia con celle ostacolo e libere

```
Funzione GeneraOstacoli(Griglia, P, S):
    if P in {SPIRALE, RIGHE, UNICO_OSTACOLO, SCACCHI} then
        // Algoritmo deterministico per il pattern scelto;
        PosizionaOstacoliDeterministicamente(Griglia, P);
    else if P in {DENSITÀ_BASSA, DENSITÀ_MEDIA, DENSITÀ_ALTA}
     then
        InizializzaRandom(S);
        rapporti=calcolaRapportiOstacoliBaseRandom();
        calcolaQuantitàTarget(densità target, rapporti);
        ScegliCellaRandom();
        PosizionaOstacoli();
        return Griglia;
```

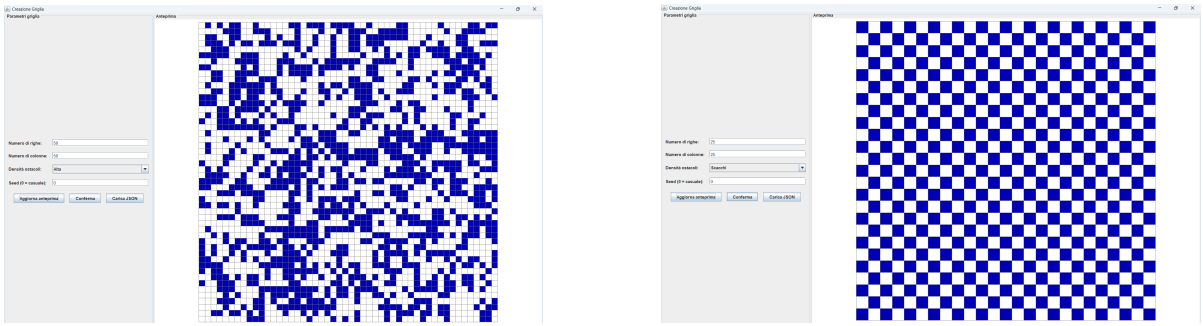


Figura 4: Esempio di generazione di griglia con pattern deterministico e randomico.

3 Compito 2: Calcolo di Contesto, Complemento e Frontiera

Una volta creata la griglia, l'utente accede a una schermata in cui è possibile selezionare una qualsiasi **cella libera** per ottenere il **contesto** e il **complemento** di quella cella, evidenziati tramite due colori distinti.

Calcolo del Contesto

Fissata la **cella d'origine**, si scansionano tutte le celle della griglia per determinare quali appartengano al suo contesto, escludendo naturalmente le celle ostacolo.

Per questa operazione ci si affida all'algoritmo `camminoLibero(mat_tipi, origine, destinazione, tipo)`, che funziona secondo la seguente logica:

1. Gestisce i casi banali: se **origine** e **destinazione** coincidono, la funzione restituisce **True**.
2. Controlla che origine e destinazione siano all'interno della griglia e non siano ostacoli; in caso contrario restituisce **False**.
3. Divide la griglia in quattro quadranti per stabilire la direzione verso la destinazione.
4. Imposta prima il **movimento diagonale** e successivamente quello assiale (orizzontale o verticale), verificando con la funzione `percorri` se il cammino raggiunge la destinazione senza incontrare ostacoli o uscire dai bordi della griglia.
5. Se il movimento diagonale ha esito positivo, viene calcolato il movimento assiale rimanente in base alla direzione precedentemente determinata. In caso di esito negativo, la destinazione non appartiene al contesto.

Calcolo del Complemento

Il **complemento** viene calcolato solo per le celle che:

- sono **libere**;
- non fanno già parte del contesto.

L'algoritmo utilizzato è analogo a quello del contesto, ma in questo caso il **movimento assiale** viene effettuato prima e quello diagonale dopo.

Pseudocodice degli algoritmi

Di seguito viene presentata una versione formale in pseudocodice dei due metodi principali:

Algorithm 2: camminoLibero(mat_tipi, origine, destinazione, tipo)

Input: mat_tipi, origine, destinazione, tipo (1=diagonale prima, 2=assiale prima)

Output: True se il cammino è libero, False altrimenti

```
1  dx = destinazione.colonna - origine.colonna;
2  dy = destinazione.riga - origine.riga;
3  if dx == 0 e dy == 0 then
4      return True // origine == destinazione
5  if origine o destinazione fuori griglia o ostacolo then
6      return False
7  stepX = segno(dx) ;
8  stepY = segno(dy) ;
9  absDx = —dx— ;
10 absDy = —dy— ;
11 diagSteps = min(absDx, absDy) ;
12 remSteps = —absDx - absDy— ;
13 if tipo == 1 then
14     // diagonale prima if percorri(mat_tipi, origine, stepX, stepY, diagSteps)
         == False then
15         return False
16     mid = origine + diagSteps passi diagonali;
17     if absDx ¿ absDy then
18         return percorri(mat_tipi, mid, stepX, 0, remSteps)
19     else
20         return percorri(mat_tipi, mid, 0, stepY, remSteps)
21 else if tipo == 2 then
22     // assiale prima if absDx ¿ absDy then
23         if percorri(mat_tipi, origine, stepX, 0, remSteps) == False then
24             return False
25         mid = origine + remSteps passi orizzontali;
26     else
27         if percorri(mat_tipi, origine, 0, stepY, remSteps) == False then
28             return False
29         mid = origine + remSteps passi verticali;
30     return percorri(mat_tipi, mid, stepX, stepY, diagSteps)
```

Algorithm 3: percorri(mat_tipi, start, stepX, stepY, steps)

Input: mat_tipi, start (riga,colonna), stepX, stepY, numero di passi
Output: True se il cammino è libero, False altrimenti

```
r, c = start.riga, start.colonna;
for i in 1..steps do
    r += stepY;
    c += stepX;
    if r o c fuori griglia then
        return False
    if mat_tipi[r][c] è ostacolo then
        return False
return True
```

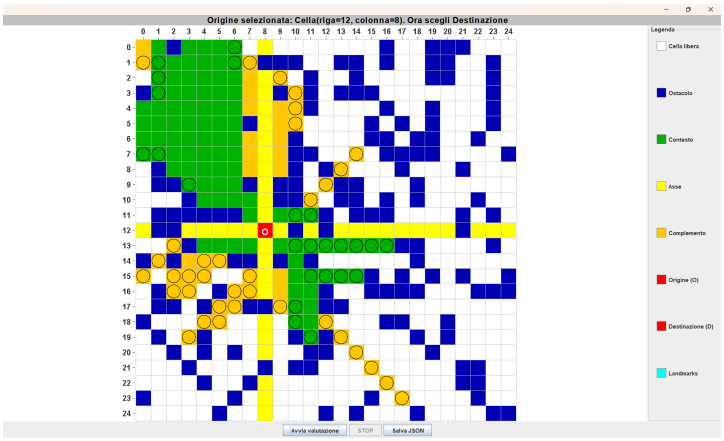


Figura 5: Esempio di evidenziazione del contesto (verde) e del complemento (arancione) di una cella O.

Riconoscimento delle Celle di Frontiera

Oltre al calcolo di contesto e complemento, è possibile individuare le **celle di frontiera** di una cella d'origine. Si definisce **frontiera** l'insieme delle celle appartenenti alla **chiusura dell'origine** e adiacenti ad almeno una **cella libera**.

Per riconoscerle, si procede nel seguente modo:

1. Si scansionano tutte le celle appartenenti alla chiusura dell'origine.
2. Per ciascuna cella c, si verifica se almeno una delle 9 celle adiacenti (verticali, orizzontali e diagonali) è libera.

3. Se almeno una cella adiacente è libera, la cella c viene contrassegnata come parte della **frontiera**.

Nota importante: il metodo presuppone che la cella c appartenga già alla chiusura (constesto o complemento) dell'origine.

Pseudocodice del metodo controllaVicini

Algorithm 4: controllaVicini(cella, griglia)

Input: cella (già appartenente alla chiusura dell'origine), griglia_tipi
Output: True se la cella è sulla frontiera, False altrimenti

```
for ogni offset dx in {-1,0,1} do
    for ogni offset dy in {-1,0,1} do
        if dx == 0 e dy == 0 then
            continua // ignora la cella stessa
        nx = cella.colonna + dx;
        ny = cella.riga + dy;
        if nx o ny fuori griglia then
            continua
        if griglia_tipi[ny][nx] è libera then
            return True
return False
```

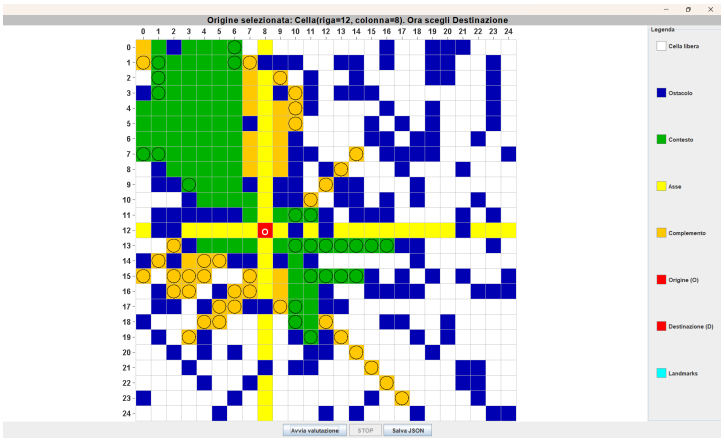


Figura 6: Esempio di evidenziazione delle celle di frontiera tramite un cerchio contenuto in alcune celle facenti parti del contesto o del complemento.

4 Compito 3: Determinazione del Cammino Minimo Base

Il **Compito Tre** si concentra sull'implementazione di un algoritmo per la determinazione del **Cammino Minimo** tra una cella di **Origine** (O) e una cella di **Destinazione** (D) all'interno di una griglia bidimensionale, che può contenere ostacoli. L'implementazione è gestita dalla classe `AnalizCam`, che espone il metodo principale `camminoMin(...)`. Questo metodo media con l'utente o le altri classi attraverso una classe controller chiamata `AgenteCamminoMinimo`, CamminoMin(...) gestisce la logica ricorsiva a mentre l'agente si occupa dell'inizializzazione, le statistiche e le interruzioni, in generale di adattare le classi esterne all'utilizzo del metodo CamminoMin(...).

Pseudocodice dell'Algoritmo Cammino Minimo

Algorithm 5: CamminoMin(griglia, O, D, compito2)

Input: griglia, origine O, destinazione D,
Output: lunghezzaMinima, sequenza di Landmark

```
1  CHECK_INTERRUZIONI();
2  camminoCorrente.aggiungi(O) g2 ← CALCOLA_CC(griglia, O);
3  frontiera ← CALCOLA_FRONTIERA(g, O, D);
4  if D ∈ contesto[g] then
5      aggiornaMigliorParziale();
6      return DLIB(O, D), ⟨(O, 0), (D, 1)⟩;
7  if D ∈ complemento[g] then
8      aggiornaMigliorParziale();
9      return DLIB(O, D), ⟨(O, 0), (D, 2)⟩;
10 if frontiera = ∅ then
11     return ∞, ⟨⟩;
12 lunghezzaMin ← ∞;
13 seqMin ← ⟨⟩;
14 foreach (F, t) ∈ frontiera do
15     lF ← DLIB(O, F);
16     if lF ¿ DLIB(O, F) then
17         continue ;
18     lFD, seqFD ← CAMMINO_MIN(g2, F, D);
19     lTot ← lF + lFD;
20     if lTot ¡ lunghezzaMin then
21         lunghezzaMin ← lTot;
22         seqMin ← COMPATTA(⟨(O, 0), (F, t)⟩, seqFD);
23         camminoCorrente.rimuovi(ultimo)
24 return lunghezzaMin, seqMin;
```

Flusso di Esecuzione e Passi Fondamentali

La logica ricorsiva di `calcoloCamminoMin` segue i seguenti passaggi principali:

1. **Preparazione alla Ricorsione:** all'inizio di ogni chiamata ricorsiva, il livello di ricorsione viene incrementato e viene eseguito un controllo per eventuali interruzioni tramite `checkInterruzione()`.

2. **Creazione della nuova configurazione della Griglia con Origine:** Questa griglia contiene la **chiusura** dell'origine (nuova), assegnando a ciascuna cella il valore di **Contesto** o **Complemento**.

3. **Caso Base – Destinazione Raggiunta:** l'algoritmo verifica se la cella di destinazione D appartiene al contesto o al complemento della cella d'origine. Se la condizione è soddisfatta, significa che la destinazione è direttamente raggiungibile dall'origine, e viene costruito e restituito un cammino dalla cella O alla cella D.

4. **Estrazione della Frontiera:** se il caso base non è soddisfatto, l'algoritmo individua la **frontiera** della griglia, ovvero l'insieme delle celle che rappresentano i possibili punti di espansione successivi.

5. **Caso Frontiera Vuota – Destinazione Irraggiungibile:** se la frontiera è vuota, significa che non esistono celle esplorabili dalla posizione corrente per raggiungere la destinazione. In questo scenario, il cammino è considerato irraggiungibile e viene restituita una lunghezza infinita con una sequenza di Landmark vuota.

6. **Esplorazione Ricorsiva della Frontiera:** per ogni cella F nella frontiera:

   - la griglia corrente viene copiata e modificata; la chiusura della cella corrente viene aggiunta come ostacolo per prevenire cicli. Questa griglia (g2) viene utilizzata nei calcoli successivi;
   - viene calcolata nuovamente la frontiera;
   - viene applicata una **condizione di ottimizzazione** per decidere se procedere con la ricorsione su F, potando i rami di ricerca che non possono migliorare la lunghezza minima trovata fino a quel momento;
   - se la condizione è soddisfatta, viene effettuata la chiamata ricorsiva a `calcoloCamminoMin(g2, F, D)`, ottenendo il cammino minimo dalla cella F alla destinazione nella nuova griglia;
   - la lunghezza del cammino parziale lF viene sommata alla lunghezza ricorsiva lFD. Se la somma $lTot$ è minore della lunghezza minima corrente, sia `lunghezzaMin` sia `seqMin` vengono aggiornati. In caso di cammini con lunghezza identica, viene mantenuto quello con scoperto per primo.
   - se la condizione di ottimizzazione non è soddisfatta, viene incrementato un contatore statistico (`stats.incrementaIterazioniCondizione()`), indicando che il ramo di ricerca è stato potenzialmente potato.

7. **Restituzione del Risultato:** al termine del ciclo su tutte le celle della frontiera, l'algoritmo restituisce il cammino minimo trovato e il livello di ricorsione viene decrementato.

8. **Fine Esecuzione:** il calcolo si conclude quando, al primo livello di ricorsione, tutte le celle di frontiera esplorabili sono state valutate. Il risultato finale rappresenta il **cammino minimo** tra Origine e Destinazione.

4.1 Stop anticipato e Gestione dei Risultati Sub-Ottimi

In alcuni scenari, l'algoritmo del Cammino Minimo potrebbe non completare la sua esecuzione entro il tempo disponibile o potrebbe essere interrotto dall'utente. Per gestire questi casi, è possibile implementare un meccanismo di **stop anticipato** che consente di accedere comunque alla migliore soluzione trovata fino a quel momento.

Concetto Chiave

A causa della natura **ricorsiva** dell'algoritmo, il cammino ottimo completo dall'origine alla destinazione viene costruito solo alla terminazione della procedura. Tuttavia, è possibile mantenere una copia dei **cammini sub-ottimi** ogni volta che l'algoritmo raggiunge la destinazione, confrontandoli con il cammino migliore precedentemente trovato. In questo modo, anche se l'algoritmo viene interrotto, si può restituire il cammino minimo *sub-ottimo* esplorato fino a quel momento.

Meccanismo di Salvataggio Intermedio

Il cammino corrente viene aggiornato dinamicamente durante l'esplorazione della griglia:

- ogni volta che si visita una nuova cella, questa viene aggiunta in coda al **camminoCorrente.aggiungi(O)**;
- quando si risale l'albero dei percorsi (backtracking), l'ultima cella viene rimossa dalla coda:**camminoCorrente.rimuovi(ultimo)**;
- ogni volta che si raggiunge la cella di destinazione, il cammino corrente viene confrontato con il **risultato intermedio** salvato:**aggiornaMigliorParziale()**;
- se la lunghezza del cammino corrente è minore di quella del risultato intermedio, quest'ultimo viene aggiornato.

Pseudocodice per la Gestione dei Risultati Sub-Ottimi

Algorithm 6: Salvataggio del Risultato Intermedio

Input: camminoCorrente: lista di celle visitate, risultatoIntermedio: cammino migliore finora

Output: Aggiornamento di risultatoIntermedio se camminoCorrente è migliore

```
somma ← 0;
for i = 0 to camminoCorrente.size()-2 do
    somma ← somma + dlib(camminoCorrente[i], camminoCorrente[i+1]);
if somma ¡ risultatoIntermedio.lunghezza then
    risultatoIntermedio.lunghezza ← somma;
    copiaLista(camminoCorrente, risultatoIntermedio.seq);
```

Integrazione nell'Algoritmo Base

Per gestire lo stop,la classe che espone l'algoritmo principale (`CamminoMin`) deve essere contenere la variabile globale descritta prima.

Vantaggi: questa strategia permette di ottenere un cammino sub-ottimo valido anche se l'algoritmo non ha completato tutte le esplorazioni, rendendo l'implementazione più robusta in scenari real-time o con vincoli di tempo.

5 Presentazione del Risultato

Il risultato prodotto dall'algoritmo del Cammino Minimo è rappresentato da un oggetto di tipo **RisultatoCammino**.

Per il corretto funzionamento dell'algoritmo, sono fondamentali soltanto due informazioni:

- la **lunghezza** totale del cammino minimo trovato;
- la **sequenza di celle** che costituisce il percorso dall'Origine alla Destinazione.

Tutti gli altri campi presenti nell'oggetto, come ad esempio il tempo di calcolo, la griglia di riferimento, i contatori statistici o informazioni sulla convergenza, rappresentano dati **aggiuntivi** utili per la visualizzazione, l'analisi e la reportistica. Queste informazioni vengono integrate nel risultato in una fase successiva, esterna all'algoritmo principale, e servono principalmente a fornire all'utente una visione più completa delle prestazioni e dello stato del calcolo.

Per una descrizione completa e dettagliata su come trattare, visualizzare e interagire con i risultati, si rimanda al **manuale d'uso del software**, che fornisce tutte le indicazioni operative e le modalità di interpretazione dei dati aggiuntivi.

5.1 Ottimizzazioni dell'Algoritmo Cammino Minimo

L'algoritmo del Cammino Minimo può essere significativamente migliorato introducendo alcune ottimizzazioni che ne riducono i tempi di esecuzione e aumentano l'efficienza complessiva. Le principali strategie implementate sono le seguenti.

1. Condizione più stringente per la potatura

La prima ottimizzazione consiste nel rafforzare la condizione alla riga 16 dello pseudocodice originale. Invece di utilizzare la condizione:

$$l_F > lunghezzaMin$$

si utilizza:

$$l_F + \text{dLib}(F, dest) > lunghezzaMin$$

Questa modifica permette di potare preventivamente rami dell'albero dei percorsi che non possono produrre un cammino più breve, riducendo drasticamente lo spazio di ricerca.

2. Riordino della Frontiera

Un'altra ottimizzazione rilevante riguarda l'**ordinamento delle celle di frontiera** prima di iterare su di esse. Questa funzionalità è gestita dalla classe `FrontieraStrategy`. Le celle vengono ordinate in base alla distanza non decrescente dalla destinazione:

Utils.distanzaLiberaTra(F, dest)

Impatto: esplorare prima le celle più vicine alla destinazione aumenta le probabilità di trovare rapidamente un cammino minimo, riducendo le chiamate ricorsive non necessarie.

Algorithm 7: Riordina Frontiera in base alla distanza dalla Destinazione

Input: frontiera: lista di celle, destinazione: cella D
Output: frontiera ordinata

```
1 frontiera.sort(Comparator.comparingDouble(l → dlib(l, destinazione)));
2 return frontiera
```

Nota: in uno sviluppo futuro si potrebbe utilizzare una struttura dati più efficiente (ad esempio una hash map con ID univoco per cella) per ridurre lo spazio di memoria riservato, mantenendo un accesso diretto.

3. Rafforzamento della condizione globale

La terza ottimizzazione consiste nel rendere il confronto della lunghezza minima **globale**, anziché circostanziale al livello di ricorsione. In pratica, invece di confrontare solo la lunghezza parziale del cammino dal nodo corrente, si calcola una nuova lunghezza totale ogni volta che si raggiunge la destinazione. Questo approccio permette di potare in modo più aggressivo rami che non possono migliorare la lunghezza minima globale trovata fino a quel momento. E' molto semplice da calcolare basta modifcare la prima chimata chiamata all'algoritmo in **CamminoMin(griglia, origine, destinazione, 0)** , e invece modificare la chiamata ricorsiva diventa **CamminoMin(griglia, origine, destinazione, lunghezzaCorrente + dLib(origine, cellaFrontiera)**. Si necessita poi di una variabile globale al di fuori della logica di ricorsione **lunghezzaMinGlobale** che va confrontata e in caso aggiornata ogni volta che si raggiunge la destinazione. Infine il confronto di riga 16 diventa:

$$lunghezzaCorrente + l_F + \text{dLib}(F, dest) < lunghezzaMinGlobale$$

4. Utilizzo della memoria dei nodi già visitati

Per evitare ricalcoli ridondanti, è possibile memorizzare il **miglior percorso trovato per ciascun nodo**. Per farlo si utilizza una struttura di supporto: un array bidimensionale `celleVisitateMatrice` che imagazzina il percorso da una cella qualsiasi a quella di destinazione salvando la sequenza indicizzata dalle coordinate della cella di origine della stessa. In questo modo, quando la ricorsione ritorna su un nodo già visitato, è possibile riutilizzare direttamente il cammino salvato accendendo con un costo fisso.

Algorithm 8: Memorizzazione dei Cammini già Visitati

```
1 Funzione inizializzaMemoriaSequenze (nr, nc):
2     celleVisitateMatrice ← nuovo array [nr][nc];
3 Funzione aggiungiAMemoria (cella, risultatoCammino):
4     celleVisitateMatrice[cella.riga][cella.colonna] ← risultatoCammino;
5 Funzione checkMemoria (cella):
6     return celleVisitateMatrice[cella.riga][cella.colonna];
```

Impatto: questa ottimizzazione riduce il numero di calcoli ricorsivi ridondanti, migliorando la velocità dell'algoritmo. Come sviluppo futuro, è possibile combinare le coordinate delle celle in un unico ID e utilizzare una **hash map** per ridurre l'occupazione di memoria mantenendo un accesso diretto efficiente.

6 Compito 4: Analisi Prestazioni

Si assuma che una Griglia abbia dimensione $n \times m$, dove n indica la larghezza e m l'altezza della Griglia. Le analisi che seguono sono state condotte considerando tutte le implementazioni del *Compito Tre*.

6.1 Analisi Spaziale

La firma del metodo principale accetta in ingresso una Griglia, una Cella di Origine e una Cella di Destinazione. La complessità spaziale iniziale risulta pertanto pari a:

$$O(n \cdot m) + O(1) + O(1) = O(n \cdot m)$$

L'algoritmo fa inoltre uso di uno **stack** per mantenere il Cammino corrente sotto forma di Landmark. Lo spazio massimo occupato dallo stack è proporzionale alla profondità massima l della ricorsione, contribuendo con un termine $O(l)$.

All'inizio di ogni chiamata ricorsiva viene creata una nuova Griglia in cui è impostata la nuova Origine. Poiché le dimensioni della Griglia rimangono invariate nel caso peggiore, tale operazione contribuisce con un ulteriore $O(n \cdot m)$ alla complessità spaziale.

Se l'algoritmo non termina immediatamente nel caso base, viene inoltre creata una struttura dati per contenere le Celle di Frontiera. Assumendo che, nel caso peggiore, tutte le Celle possano appartenere alla Frontiera, questa struttura contribuisce con $O(n \cdot m)$.

Quando la Chiusura precedente viene trasformata in Ostacolo, viene generata una nuova Griglia avente le stesse dimensioni. Di conseguenza, lo spazio occupato dalle Griglie generate durante le chiamate ricorsive è pari a:

$$O(l) \cdot O(n \cdot m)$$

Infine, ogni volta che viene individuato un nuovo potenziale Cammino Minimo, questo viene confrontato con il miglior Cammino trovato fino a quel momento. In tale fase, coesistono in memoria due Cammini, ciascuno contenente una sequenza di Landmark. Nel caso peggiore, il numero di Landmark è pari a $n \cdot m$, determinando un ulteriore contributo pari a:

$$2 \cdot O(n \cdot m)$$

Complessivamente, la complessità spaziale nel caso peggiore risulta:

$$O(n \cdot m) + O(l) + O(n \cdot m \cdot l) + 2 \cdot O(n \cdot m) = O(n \cdot m \cdot l)$$

!!!!ATTENZIONE!!!!-¿riscrivere bene: C'è anche da considerare quando viene utilizzata la memoria per salvare delle sequenze. si utilizza una matrice delle stesse dimensioni della griglia originale che nel caso peggiore contiene un cammino di lunghezza l e quindi un o(m * n* l) c'è anche da dire che in molto casi i cammini sono sottocammnini di l

Caso Migliore Nel caso migliore, la Cella di Destinazione appartiene al Contesto o al Complemento dell'Origine. In questa situazione:

- lo stack contiene solo Origine e Destinazione;
- viene comunque creata una nuova Griglia di dimensione $O(n \cdot m)$;
- il Cammino risultante contiene soltanto due Landmark;
- la profondità massima della ricorsione è $l = 1$.

Pertanto, la complessità spaziale nel caso migliore è:

$$O(n \cdot m)$$

6.2 Analisi Temporale

La creazione di una nuova Griglia avviene tramite il metodo `calcolaCC` del *Compito Due*. Tale metodo utilizza un doppio ciclo annidato sulle dimensioni della Griglia, determinando una complessità temporale pari a $O(n \cdot m)$.

L'implementazione di `calcola` influisce su questa stima:

- con l'implementazione **V0**, la rigenerazione della Griglia mantiene invariate le dimensioni, producendo una complessità $\Theta(n \cdot m)$;
- con l'implementazione **V1**, alcune Celle possono essere escluse, riducendo potenzialmente le dimensioni effettive, ma la complessità resta $O(n \cdot m)$.

Per semplicità di analisi, si assume come riferimento l'implementazione **V1**.

L'algoritmo itera quindi su tutte le Celle della Griglia, contribuendo con $O(n \cdot m)$. Successivamente, viene eseguito un ulteriore doppio ciclo su una struttura di pari dimensioni. I cicli interni operano su un massimo di 9 Celle adiacenti, costituendo un costo costante $O(1)$. Pertanto, ogni creazione di una Griglia comporta una complessità complessiva pari a:

$$O(n \cdot m)$$

Tornando all'implementazione del *Compito Tre*, assumendo che le operazioni di *push* e *pop* sullo stack abbiano costo $O(1)$, la creazione di una nuova Griglia aggiunge un ulteriore $O(n \cdot m)$.

Quando viene costruita la lista delle Frontiere, questa può essere ordinata in ordine non decrescente rispetto alla distanza dalla Destinazione. Nel caso peggiore, il numero di Celle di Frontiera è pari a $n \cdot m$:

- la trasformazione in lista richiede $O(n \cdot m)$;

- l'ordinamento, implementato in Java tramite *Dual-Pivot Quicksort*, richiede $O(n \cdot m \cdot \log(n \cdot m))$.

Nel caso peggiore, la complessità temporale dipende fortemente dal numero di Celle di Frontiera, per il quale non è possibile determinare un limite superiore basato esclusivamente sulle dimensioni della Griglia. Le sperimentazioni mostrano infatti una chiara correlazione tra il tempo di esecuzione e il numero di Celle di Frontiera.

In questo caso la memoria dei nodi già salvati non inficia sulle prestazioni infatti ha un cossto di accesso di O(1)

Caso Migliore Nel caso migliore, la Destinazione appartiene al Contesto o al Complemento dell'Origine. L'esecuzione termina quindi prima della creazione della Frontiera. Tutte le operazioni effettuate fino a questo punto hanno costo $\Omega(1)$, ad eccezione della creazione della nuova Griglia, che mantiene una complessità $\Omega(n \cdot m)$.

Di conseguenza, la complessità temporale nel caso migliore è:

$$\Omega(n \cdot m)$$

6.3 Benchmark e Campione di Test

Per l'analisi delle prestazioni dell'algoritmo, è stato creato un insieme di griglie di test rappresentative dei diversi scenari possibili.

Le configurazioni principali includono:

- Griglie di dimensioni 100×100, 25×25 e 10×10, con densità di ostacoli variabile, per valutare l'impatto della scala e della complessità spaziale.
- Per ciascuna delle altre tipologie di pattern definite, è stato realizzato almeno un esemplare con dimensione 25×25, consentendo di testare comportamenti differenti in termini di distribuzione degli ostacoli.

In tutti i casi, i punti di **Origine** e **Destinazione** sono stati scelti in modo da massimizzare la difficoltà del percorso, garantendo test impegnativi per l'algoritmo. Ogni configurazione di griglia è stata inoltre duplicata invertendo Origine e Destinazione, per verificare la simmetria e la robustezza delle prestazioni rispetto alla direzione del cammino.

7 Conclusioni

....

Sviluppi futuri

Nel corso dello sviluppo del software, l'attenzione principale è stata rivolta alla riduzione del numero di nodi esplorati nell'albero di ricorsione, privilegiando strategie algoritmiche di potatura e ottimizzazione del processo di ricerca del cammino. Di conseguenza, sono stati volutamente tralasciati approcci orientati allo sfruttamento diretto delle risorse hardware, che potrebbero costituire una naturale estensione del lavoro svolto.

In particolare, non sono state investigate tecniche di ottimizzazione legate alle prestazioni della macchina, quali l'uso esplicito della memoria cache, la parallelizzazione delle computazioni o l'impiego di strutture dati e algoritmi progettati per favorire la località dei dati. L'introduzione di tali strategie potrebbe portare a un miglioramento significativo dei tempi di esecuzione, soprattutto nel caso di griglie di grandi dimensioni o di alberi di ricerca particolarmente profondi.

Analogamente, l'ottimizzazione dello spazio occupato non è stata oggetto di un'analisi approfondita. Attualmente, la memorizzazione dei nodi già visitati avviene tramite una matrice bidimensionale indicizzata direttamente dalle coordinate della Griglia. Sebbene questa soluzione risulti semplice ed efficiente in termini di accesso, essa è potenzialmente sovradimensionata dal punto di vista della memoria utilizzata.

Un possibile sviluppo futuro consiste nella sostituzione di tale struttura con una *hash map*, ottenuta combinando le coordinate della Cella in un identificatore univoco. Questo approccio consentirebbe di ridurre sensibilmente l'occupazione di memoria, mantenendo al contempo un accesso diretto ai nodi già esplorati.

Un'ulteriore direzione di miglioramento riguarda la rappresentazione stessa della Griglia. L'utilizzo di strutture compatte, come ad esempio *bitmap* o rappresentazioni binarie, in luogo di una matrice di oggetti Cella, permetterebbe di ridurre in maniera significativa l'ingombro in memoria. Tale scelta risulterebbe particolarmente vantaggiosa nel caso di Griglie di grandi dimensioni e di alberi di ricorsione fortemente stratificati, nei quali il numero di strutture allocate cresce rapidamente.

Nel complesso, l'integrazione di queste ottimizzazioni potrebbe rendere l'algoritmo non solo più efficiente dal punto di vista computazionale, ma anche più scalabile e adatto a scenari applicativi di maggiore complessità.

8 Manuale d'installazione e utilizzo

Il presente documento costituisce il **manuale d'uso ufficiale** del software per l'analisi e il calcolo del **cammino minimo su griglie bidimensionali con ostacoli**.

Il sistema è progettato per:

- studiare algoritmi di ricerca del cammino minimo;
- confrontare differenti configurazioni e ottimizzazioni;
- analizzare le prestazioni in termini di **tempo** e **memoria**;
- supportare sia un utilizzo **interattivo (GUI)** sia **automatizzato (CLI)**.

Il software mette a disposizione:

- una **modalità batch** (CLI) per test massivi su directory di griglie;
- una **modalità grafica** per creazione, modifica e visualizzazione delle griglie;
- strumenti di **reportistica** (CSV, TXT, JSON);
- supporto per **analisi avanzata dello spazio** tramite Java Flight Recorder.

8.1 Requisiti di Sistema

- Java JDK 17 o superiore
- Sistema operativo: Windows, Linux o macOS
- Almeno 2 GB di RAM consigliati
- (Opzionale) Java Mission Control / VisualVM per analisi JFR

8.2 Avvio del Programma

Il programma viene avviato tramite la classe `Main` e supporta due modalità mutuamente esclusive:

- **Modalità batch (CLI)**
- **Modalità grafica (GUI)**

8.3 Sintassi generale

```
java Main <opzione> [argomenti]
```

Opzioni disponibili

Opzione	Descrizione
`-b <directory>`	Avvia il programma in modalità batch (CLI), leggendo tutte le griglie JSON presenti nella directory indicata.
`-g`	Avvia il programma in modalità grafica (GUI).

8.4 Modalità Batch (CLI)

La modalità batch è progettata per eseguire **test automatici su molte griglie** senza intervento manuale.

Avvio

```
java Main -b ./griglie/
```

Il programma:

1. legge tutti i file `.json` presenti nella directory;
2. supporta sia singole griglie che array di griglie;
3. carica tutte le griglie valide in memoria;
4. avvia il controller CLI.

Formato dei file JSON

Ogni file può contenere:

- una singola griglia;
- un array di griglie.

Le griglie includono:

- dimensioni;
- matrice degli ostacoli;
- origine e destinazione (opzionali).

Inserimento dei Parametri

Durante l'esecuzione CLI, l'utente viene guidato nell'inserimento dei parametri:

Limite di tempo

- espresso in millisecondi;
- limite massimo di esecuzione per ogni griglia;
- se superato, l'algoritmo viene interrotto.

Stringa di opzioni booleane

L'utente inserisce una stringa di 5 caratteri:

`RCMVS`

Ogni posizione rappresenta un'opzione:

Posizione	Simbolo	Significato
1	R	Riordino della frontiera
2	C	Condizione forte
3	M	Memoria delle sequenze
4	V	Versione ottimizzata
5	S	Analisi dello spazio (JFR)

Il simbolo - indica disattivazione.

Nota importante: Il funzionamento interno e teorico di ciascuna opzione è descritto in dettaglio nella **Sezione ??**.

Una volta terminata l'esecuzione si entra in un secondo sotto menu che chiederà all'utente di decidere se continuare con una nuova esecuzione, oppure uscire.

8.5 Output della Modalità Batch

Report CSV

Al termine dell'esecuzione viene generato un file:

`report.csv`

Interpretazione del file CSV generato

Il sistema, al termine dell'esecuzione in modalità batch, produce un file di report denominato `report.csv` che contiene l'aggregato di tutti i risultati ottenuti durante l'analisi delle griglie caricate. Il file è strutturato secondo il formato CSV (*Comma-Separated Values*) con intestazione e valori separati da virgola. Ogni riga del file rappresenta i risultati ottenuti per una singola griglia testata.

Il contenuto del CSV viene generato dal metodo `generateReport` della classe `GriglieTester`. L'intestazione del file contiene i seguenti campi:

1. **modalità**: indica la modalita di esecuzione dell'algoritmo, quali sono le ottimizzazioni attive. Si presenta nello stesso formato della richiesta dei parametri. Alcuni esempi: "—-" è il caso base, "F—" è attiva ottimizzazione di riordino frontiera, "F-M-" riordino frontiera e memoria dei percorsi attivi, etc.

2. **righe**: indica il numero di righe della griglia testata. Questo valore è fondamentale per comprendere la dimensione verticale della griglia e, indirettamente, la complessità computazionale del calcolo del cammino minimo, poiché griglie più grandi richiedono un numero maggiore di iterazioni dell'algoritmo di ricerca.

3. **colonne**: rappresenta il numero di colonne della griglia. Analogamente al campo `righe`, fornisce informazioni sulla dimensione orizzontale della griglia. L'accoppiamento di righe e colonne consente di calcolare l'area complessiva della griglia, utile per confronti tra test su griglie di dimensioni diverse.

4. **tipologia**: specifica il tipo di configurazione della griglia, come ad esempio "DENSITA_BASSA", "DENSITA_MEDIA", "SPIRALE", "SCACCHI", "RIGHE", "UNICO_OSTACOLO", o "PERSONALIZZATA". Questo campo permette di distinguere tra griglie generate automaticamente secondo pattern predefiniti e griglie create manualmente dall'utente, e consente di correlare le prestazioni dell'algoritmo con la complessità della disposizione degli ostacoli.

5. **d_lib**: rappresenta la distanza libera stimata (*d_lib*) tra origine e destinazione, calcolata dall'algoritmo come parte del risultato analitico preliminare. Tale valore viene espresso in numeri reali con tre cifre decimali e riflette un'indicazione della "libertà" di movimento all'interno della griglia. Maggiore è il valore, più spazio libero esiste tra ostacoli e più facile potrebbe essere trovare un cammino ottimale.

6. **dest_ragg**: indica se la destinazione è raggiungibile dalla posizione di partenza nella griglia. Questo campo assume valori binari:

   - 1: la destinazione è raggiungibile.
   - 0: la destinazione non è raggiungibile a causa della disposizione degli ostacoli.

Questa informazione è essenziale per interpretare i risultati di convergenza e lunghezza del cammino: se la destinazione non è raggiungibile, il resto dei valori potrebbe risultare incompleto o non significativo.

7. **convergenza**: indica se l'algoritmo ha completato la ricerca del cammino minimo entro i vincoli temporali impostati. Il campo assume i valori:

   - 1: il calcolo ha raggiunto la convergenza completa.
   - 0: il calcolo è stato interrotto anticipatamente (per limite di tempo o per intervento dell'utente).

   Per maggiori dettagli sulle condizioni di interruzione e sulle ottimizzazioni utilizzate, si rimanda alla sezione **??**.

8. **lunghezza**: rappresenta la lunghezza del cammino minimo trovato dall'agente. Questo valore numerico indica il numero di passi necessari per raggiungere la destinazione dalla posizione di partenza, considerando il percorso calcolato dall'algoritmo. Il campo è espresso in numeri interi o decimali, a seconda del tipo di calcolo implementato.

9. **tempo_ms**: indica il tempo impiegato dall'algoritmo per calcolare il cammino minimo, espresso in millisecondi. Questo valore è utile per confrontare le prestazioni su griglie di diversa dimensione o complessità. L'analisi dei tempi permette di valutare l'efficacia delle ottimizzazioni implementate nel motore di calcolo del percorso.

10. **istr16**: contatore interno relativo a un particolare tipo di istruzione o operazione effettuata dall'agente durante il calcolo del cammino. Per il significato specifico, si rimanda alla documentazione delle ottimizzazioni descritte nella sezione **??**.

11. **cont_frontiera**: indica il numero di nodi aggiunti alla frontiera di ricerca durante l'esplorazione della griglia. La frontiera rappresenta l'insieme delle celle candidate per il passo successivo del cammino. Questo valore consente di analizzare l'efficienza della gestione della frontiera e l'impatto delle eventuali ottimizzazioni sulla riduzione dei nodi visitati.

12. **nodi_salvati**: rappresenta il numero di nodi che sono stati memorizzati durante l'esecuzione dell'algoritmo per eventuali controlli o ripercussioni successive. Questo campo è correlato all'ottimizzazione della memoria e alla gestione delle sequenze di cammino, rimandando anche qui alla sezione **??**.

13. **memo_usata**: indica la quantità di memoria heap utilizzata durante l'esecuzione del calcolo del percorso. Il valore è espresso in kilobyte (KB) e consente di valutare

l'impatto delle ottimizzazioni legate alla gestione della memoria, come ad esempio l'uso della memoria per sequenze parziali o versioni ottimizzate degli agenti.

14. **landmarks**: contiene la sequenza completa delle celle percorse dal cammino minimo trovato, rappresentata come stringhe separate dal carattere `|`. Questo campo è principalmente di supporto e permette di ricostruire il cammino per visualizzazioni o ulteriori analisi. Poiché la sequenza può essere molto lunga, viene generalmente utilizzata solo per debugging o per generare mappe grafiche della soluzione.

Nota importante: alcuni campi, come `istr16`, `cont_frontiera` e `nodi_salvati`, sono direttamente influenzati dalle scelte di ottimizzazione del calcolo del cammino minimo, come il riordino della frontiera, l'uso della memoria per sequenze parziali, la versione ottimizzata dell'algoritmo, e altre strategie di miglioramento delle prestazioni. Per una spiegazione dettagliata delle implicazioni di ciascun parametro e del loro effetto sui valori riportati nel CSV, consultare la sezione **??** del manuale.

In sintesi, il file `report.csv` fornisce una panoramica completa delle caratteristiche delle griglie, delle prestazioni dell'agente e delle metriche di memoria e calcolo, consentendo all'utente di effettuare confronti sistematici tra test differenti e valutare l'impatto delle ottimizzazioni implementate.

Se l'opzione `S` è attiva:

- viene generato un file `.jfr` per ogni griglia;
- i file sono apribili con Java Mission Control o VisualVM;
- consentono analisi dettagliate delle allocazioni di memoria.

Il file contiene una riga per ogni griglia analizzata.

File JFR

Se l'opzione `S` è attiva:

- viene generato un file `.jfr` per ogni griglia;
- i file sono apribili con Java Mission Control o VisualVM;
- consentono analisi dettagliate delle allocazioni di memoria.

8.6 Modalità Grafica (GUI)

La modalità grafica offre un ambiente interattivo per:

- creare griglie;

- modificarle manualmente;
- selezionare origine e destinazione;
- visualizzare il cammino;
- salvare e caricare file.

Creazione della Griglia

La finestra **Creatore Griglia** consente di:

- scegliere righe e colonne (max 100x100);
- selezionare il pattern di ostacoli;
- impostare un seed casuale;
- creare griglie personalizzate manualmente;
- caricare griglie da JSON.

Pattern Disponibili

- Densità bassa
- Densità media
- Densità alta
- Spirale
- Scacchiera
- Righe
- Unico ostacolo
- Personalizzata

Per consultate la tipologia di griglia consultare "la sezione 1 generazione griglia"

Interazione con la Griglia

Nella finestra principale:

- clic sinistro per selezionare origine (O);
- secondo clic per destinazione (D);
- clic successivo resetta l'origine.

Avvio del Calcolo

Alla pressione di **Avvia valutazione**:

- si apre una finestra di configurazione;
- è possibile impostare le stesse opzioni della CLI;
- l'esecuzione avviene in background;
- il pulsante **STOP** consente l'interruzione.

Riepilogo Finale

Al termine:

- viene mostrato un riepilogo testuale;
- è possibile salvarlo in formato `.txt`.

8.7 Salvataggio e Caricamento

Salvataggio JSON

Le griglie possono essere salvate in formato JSON, includendo:

- ostacoli;
- origine e destinazione;
- dimensioni.

Caricamento JSON

Il sistema:

- valida le dimensioni;
- carica automaticamente la griglia;
- aggiorna l'anteprima.

8.8 Ottimizzazioni dell'Algoritmo

Questa sezione descrive in modo teorico e concettuale le **ottimizzazioni disponibili** durante il calcolo del cammino minimo.

Tutte le opzioni selezionabili in CLI e GUI fanno riferimento a questa sezione.

Riordino della Frontiera

Ottimizza l'esplorazione dei nodi migliorando la priorità di espansione.

Condizione Forte

Introduce criteri più restrittivi per la convergenza del cammino.

Memoria delle Sequenze

Consente il riuso di sequenze parziali già esplorate.

Versione Ottimizzata

Attiva implementazioni alternative più efficienti in termini di costo computazionale.

Analisi dello Spazio

Utilizza Java Flight Recorder per monitorare:

- allocazioni in heap;
- crescita della memoria;
- hotspot di allocazione.

Nota: Per evitare ridondanze, il comportamento dettagliato di ciascuna ottimizzazione non viene ripetuto nelle sezioni operative, ma va sempre riferito a questa sezione.

Elenco delle figure

1 Esempio di griglia con pattern a spirale. 5
2 Esempio di griglia personalizzata definita dall'utente. 5
3 Esempi di pattern automatici. 6
4 Esempio di generazione di griglia con pattern deterministico e randomico. 9
5 Esempio di evidenziazione del contesto (verde) e del complemento (arancione) di una cella O. 13
6 Esempio di evidenziazione delle celle di frontiera tramite un cerchio contenuto in alcune celle facenti parti del contesto o del complemento. 14

Elenco delle tabelle

Elenco dei listati